UDACITY

Part of Accenture

Vulnerable Web Application

Samuel H. Mariam

26 June 2026

VulnWebApp (VWA) Security Report

Code Revision	1.0.0.0
Company	USociety
Report	VWA260626
Author	Samuel H. Mariam
Date	26 June 2026

Manual web application security assessment

Section One: Static Code Scan

Issue: B106

```
>> Issue: [B106:hardcoded_password_funcarg] Possible hardcoded
password: 'mysecurepassword'
   Location: SampleCode/init_db.py:14
13      def open(self):
14          self.conn = psycopg2.connect(user = "webappuser",
15                                       password = "mysecurepassword",
16                                       host = "localhost",
17                                       port = "5432",
18                                       database = "website")
19          self.cursor = self.conn.cursor()
```

Severity:	*High*
OWASP TOP 10 reference:	*[OWASP 2021 A07: Identification and Authentication Failures]*
Remediation recommendation	
[Remove the hardcoded database password from the source code. Store database credentials in a secure secrets management system or environment variables, such as a cloud secrets manager, HashiCorp Vault, Docker/Kubernetes secrets, or protected deployment environment variables.]	

Issue: B108

```
>> Issue:  [B108:hardcoded_tmp_directory] Use of a hardcoded
temporary directory.
Location: SampleCode/temp_file.py:19

18 def createTempFile(data):
19     temp_path = "/tmp/tempfile.txt"
20     with open(temp_path, 'w') as temp_file:
21         temp_file.write(data)
 22     return temp_path
```

Severity:	*[Medium]*
OWASP TOP 10 reference:	*[OWASP 2021 A04: Insecure Design]*
Remediation recommendation	
[Do not use a hardcoded, predictable filename in /tmp. Use Python's secure temporary file utilities instead.]	

Issue: B303

```
>> Issue: [B303:blacklist] Use of insecure MD2, MD4, MD5, or SHA1
hash function.
   Location: SampleCode/create_customer.py:23
22          self.email = email
23          self.password =
hashlib.md5(password.encode('utf-8')).hexdigest()
24          self.banner = safestring.mark_safe(banner)
```

Severity:	*[High]*
OWASP TOP 10 reference:	*[OWASP 2021 A02: Cryptographic Failures]*

Remediation recommendation

*[*Replace MD5 password hashing with a modern password hashing algorithm designed for credential storage.

Recommended options include:

1. **Argon2id** — preferred modern password hashing algorithm.
2. **bcrypt** — widely supported and acceptable.
3. **PBKDF2-HMAC-SHA256** — acceptable when configured with a strong iteration count.

]

Issue: B311

```
>> Issue: [B311:blacklist] Standard pseudo-random generators are
not suitable for security/cryptographic purposes.
   Location: SampleCode/init_db.py:40
39          letters = string.ascii_lowercase
40          result_str = ''.join(random.choice(letters) for i in
range(length))
41          return result_str
```

Severity:	*[**false positive**]*
OWASP TOP 10 reference:	*[OWASP 2021 A02: Cryptographic Failures]*
Remediation recommendation	
[The random module is not designed for security-sensitive values. If the value is security-sensitive, replace Python's random module with the secrets module, which is designed for cryptographic use.] *This may be the project's **false positive** if the function is only used to generate non-sensitive test data, sample names, dummy strings, or database seed values during development.*	

Issue: B320

```
>> Issue: [B320:blacklist] Using lxml.etree.fromstring to parse
untrusted XML data is known to be vulnerable to XML attacks.
Replace lxml.etree.fromstring with its defusedxml equivalent function.
   Location: SampleCode/fix_customer_orders.py:11
10    def customerOrdersXML():
11          root = lxml.etree.fromstring(xmlString)
12          root = fromstring(xmlString)
```

Severity:	*[High]*
OWASP TOP 10 reference:	*[OWASP 2021 A05: Security Misconfiguration]*
Remediation recommendation	
[Parsing untrusted XML with unsafe XML parsers can expose the application to XML-based attacks. ***Replace unsafe XML parsing with a hardened XML parser from*** *defusedxml]*	

Issue: B603

```
>> Issue: [B603:subprocess_without_shell_equals_true] subprocess
call - check for execution of untrusted input.
   Location: SampleCode/onLogin.py:8
7          def process(self, user, startupcmd):
8              p = subprocess.Popen([startupcmd],
stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
9              r = p.communicate()[0]
```

Severity:	*[High]*
OWASP TOP 10 reference:	*[OWASP 2021 A03: Injection]*
Remediation recommendation	

[Avoid passing user-controlled values into subprocess.Popen()]

***Preferred remediation**:*

1. *Remove the subprocess call if it is not strictly required.*
2. *Replace dynamic command execution with internal application logic.*
3. *If external execution is required, use a strict allowlist of approved commands.*
4. *Use fixed absolute paths instead of user-supplied command names.*

Issue: B703

```
>> Issue: [B703:django_mark_safe] Potential XSS on mark_safe
function.
   Location: SampleCode/create_customer.py:24
23          self.password =
hashlib.md5(password.encode('utf-8')).hexdigest()
24          self.banner = safestring.mark_safe(banner)
25
```

Severity:	*[High]*
OWASP TOP 10 reference:	*[OWASP 2021 A03: Injection]*
Remediation recommendation	

[Do not use mark_safe() on untrusted input]

Additional best practices:

1. *Avoid mark_safe() for customer-controlled content.*
2. *Sanitize rich-text fields with a strict allowlist.*
3. *Reject dangerous tags such as <script>, <iframe>, <object>, and event handlers such as onclick.*
4. *Add Content Security Policy headers to reduce XSS impact.*

Section Two:

Assess the Web Application

Section Three: Security Report

VWA26062601 - Broken Authentication - High

Vulnerability Exploited:	*Broken Authentication*
Severity:	*High*
OWASP TOP 10 reference:	*A07:2021 - Identification and Authentication Failures*

Vulnerability Explanation

The login endpoint accepts repeated username and password attempts without effective rate limiting, progressive delay, temporary lockout, or multi-factor authentication. Using the provided bruteforce.py script and supplied username and password lists, repeated requests identified valid credentials and allowed access to the authenticated dashboard.

Recommendations

- Rate-limit failed logins by account, source address, and session.
- Apply progressive delays or temporary lockout after repeated failures.
- Require multi-factor authentication for administrator accounts.
- Log and alert on abnormal authentication activity while returning generic failure messages.

VWA26062601 - Broken Authentication - Steps

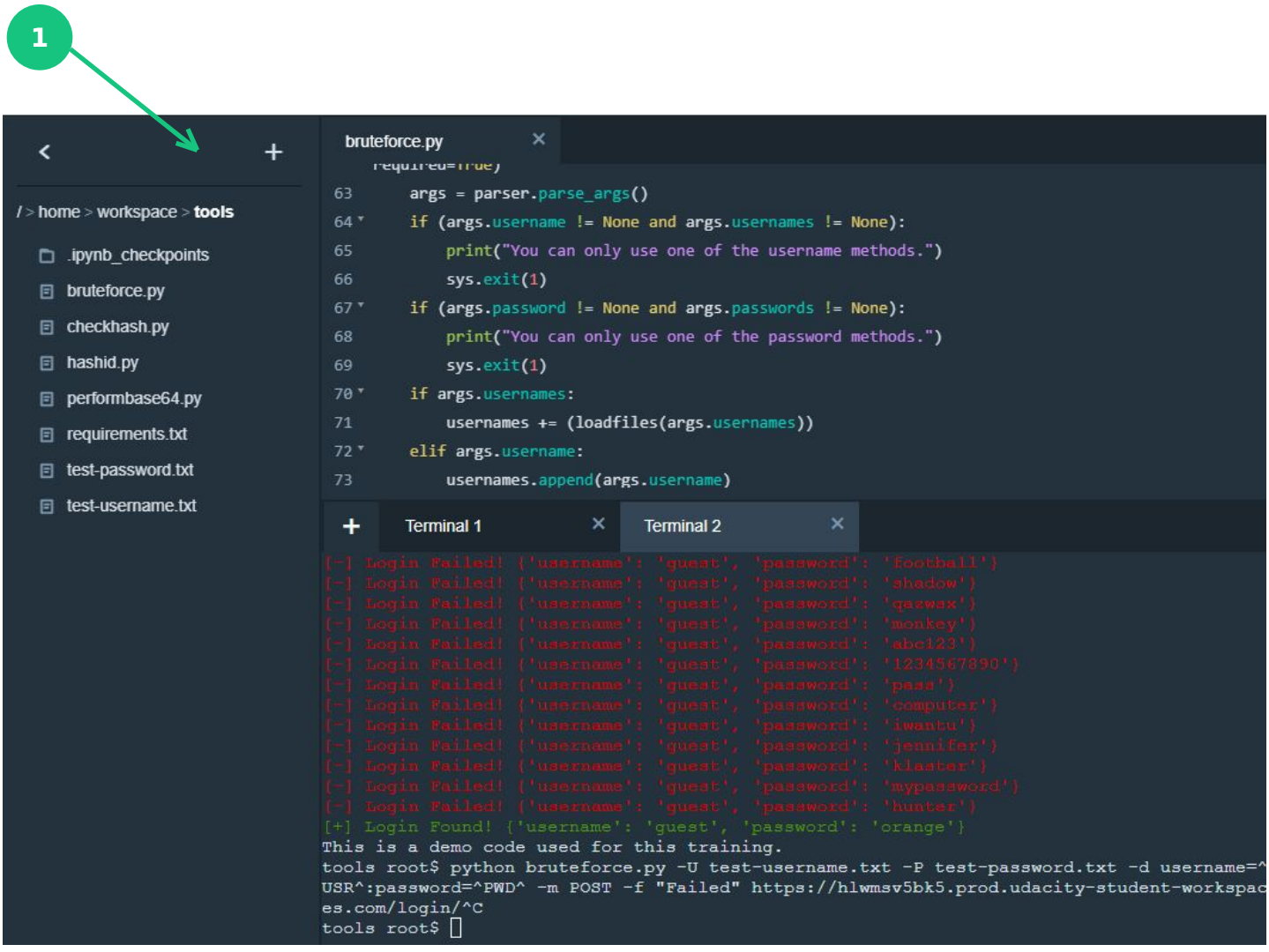
Steps to Reproduce

1. Open the application login page in a browser.
2. Run the provided bruteforce.py script with test-username.txt and test-password.txt against the login form.
3. Observe the repeated attempts and record the credential pair for which the response no longer contains the failure marker.
4. Enter the discovered credentials in the normal login form.
5. Confirm that the authenticated dashboard loads.

Evidence reference: Evidence pages 1-2 show the permitted script execution and successful authenticated dashboard.

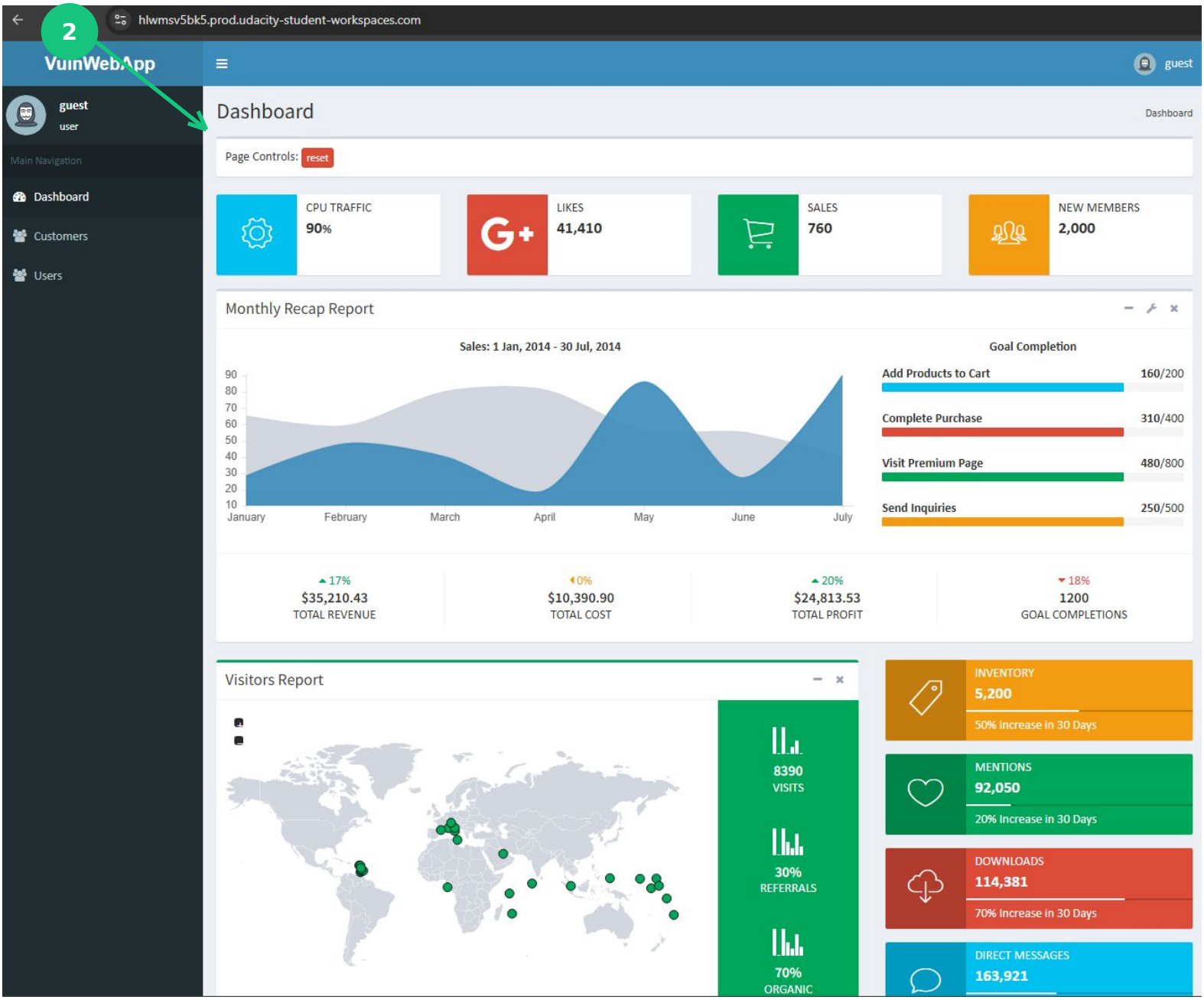
Broken Authentication

EVIDENCE 1: bruteforce.py tests the supplied credential lists



Broken Authentication

EVIDENCE 2: Valid credentials open the authenticated dashboard



VWA26062602 - Cross-Site Scripting (XSS) - High

Vulnerability Exploited:	*Cross-Site Scripting (XSS)*
Severity:	*High*
OWASP TOP 10 reference:	*A03:2021 - Injection*

Vulnerability Explanation

The profile messaging feature stores user-controlled message content and later renders it as HTML without context-appropriate output encoding. A logged-in user can store an HTML element containing an event handler; the JavaScript executes when the affected profile page loads.

Recommendations

- Render user messages as text with context-aware output encoding.
- Do not insert untrusted content with raw HTML APIs such as .append().
- If rich text is required, sanitize it server-side with a strict allowlist.
- Use a restrictive Content Security Policy as defense in depth.

VWA26062602 - Cross-Site Scripting (XSS) - Steps

Steps to Reproduce

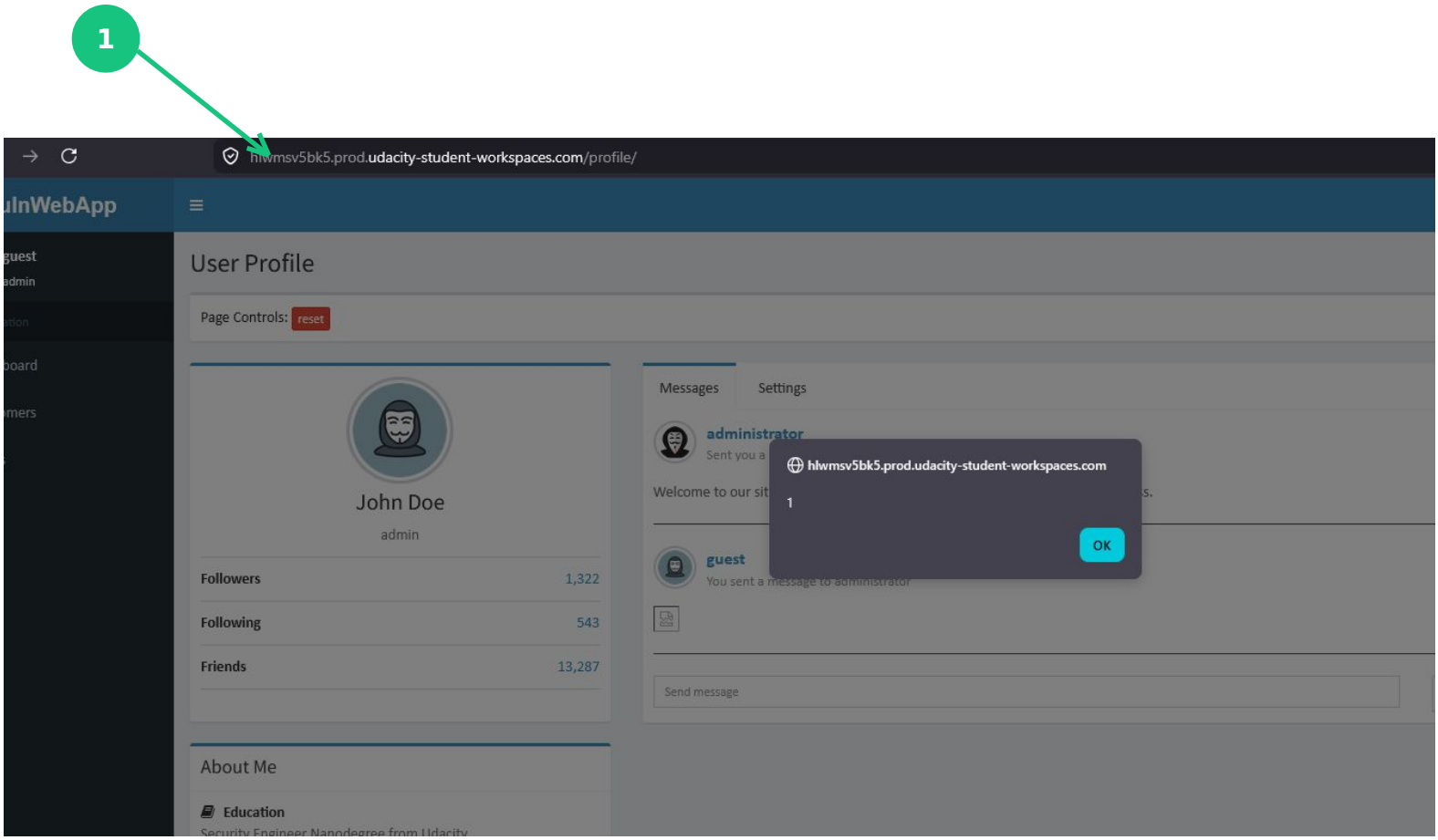
1. Log in as a normal user and open the Profile page.
2. Use the messaging form to send <img src=x onerror=alert(1)> to the test profile.
3. Reload the profile that displays the stored message.
4. Confirm that the browser executes the payload and displays the alert dialog.

Evidence reference: The annotated evidence page shows JavaScript execution in the affected profile.

Assessment used the supplied scripts, browser, and browser Developer Tools only.

Cross-Site Scripting (XSS)

EVIDENCE 1: Stored message executes JavaScript in the profile



VWA26062603 - Broken Access - Customer API - High

Vulnerability Exploited:	*Broken Access - Customer API*
Severity:	*High*
OWASP TOP 10 reference:	*A01:2021 - Broken Access Control*

Vulnerability Explanation

The customer page correctly denies a normal user, but the backing customer API returns customer records when requested directly with the same session. Authorization is enforced in the user interface but not consistently at the server endpoint, allowing a normal user to retrieve restricted data.

Recommendations

- Enforce server-side authorization on every customer page and API endpoint.
- Deny access by default and centralize role checks in tested middleware or decorators.
- Return only records and fields that the authenticated user is authorized to view.
- Add negative authorization tests for normal, unauthenticated, and cross-user sessions.

VWA26062603 - Broken Access - Customer API - Steps

Steps to Reproduce

1. Log in as a normal user.

2. Open https://hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/ and observe the authorization-denied message.

3. Using the same browser session, open https://hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/id/.

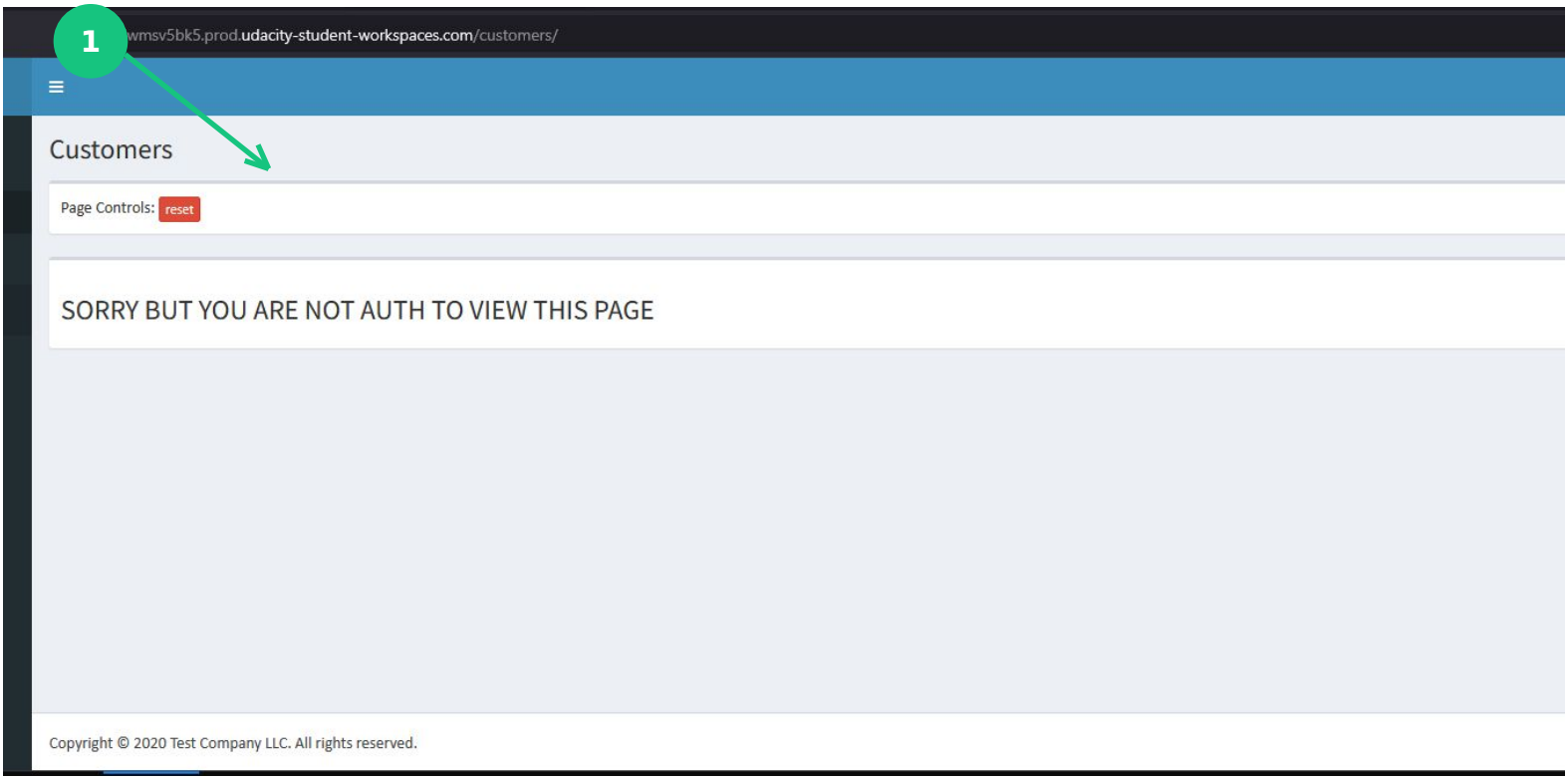
4. Confirm that the API returns restricted customer records despite the user's non-administrator role.

Evidence reference: Evidence page 1 shows the expected denial; evidence page 2 shows the API bypass.

Assessment used the supplied scripts, browser, and browser Developer Tools only.

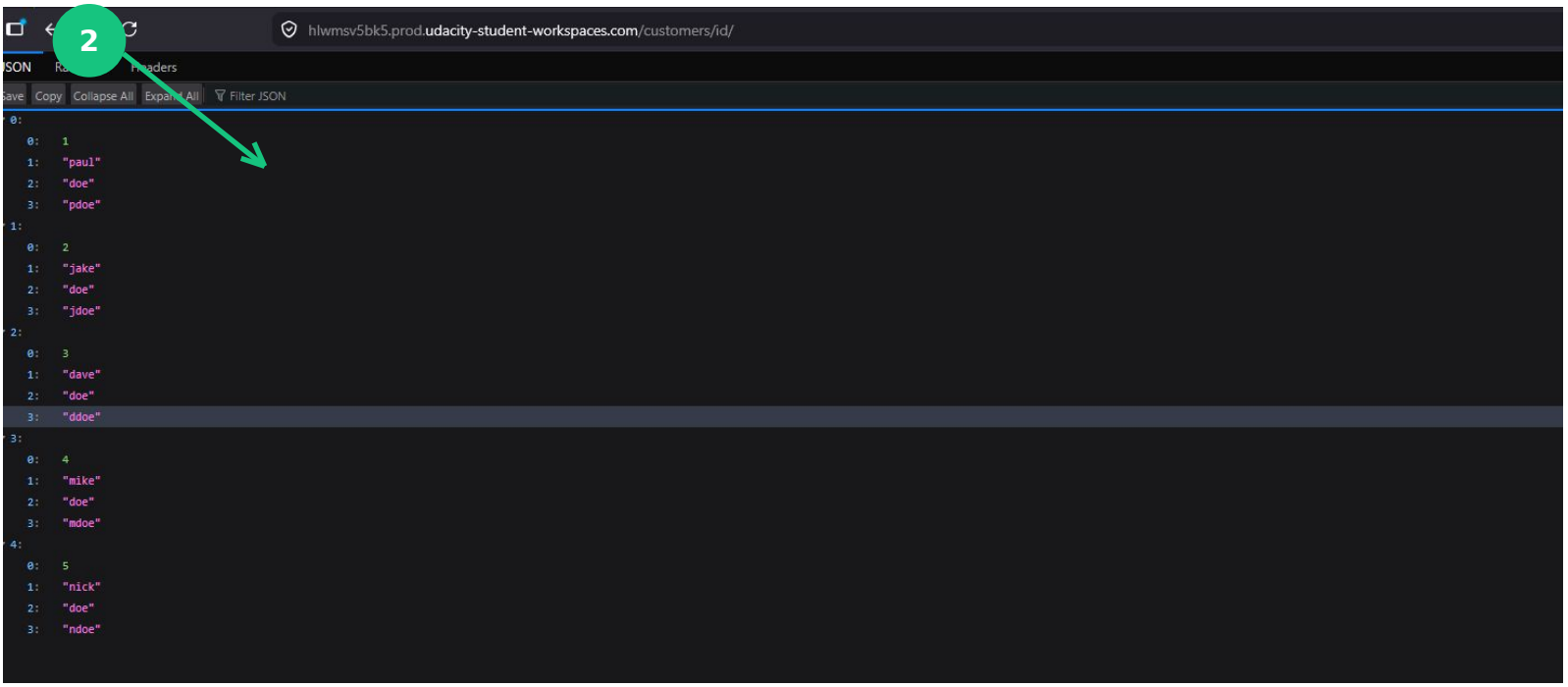
Broken Access

EVIDENCE 1: Normal user is denied by the Customers page



Broken Access

EVIDENCE 2: Direct customer API request returns restricted records



VWA26062604 - Cookie Role Tampering - Critical

Vulnerability Exploited:	*Cookie Role Tampering*
Severity:	*Critical*
OWASP TOP 10 reference:	*A01:2021 - Broken Access Control*

Vulnerability Explanation

The application trusts the client-controlled authInfo cookie to determine the user's role. Because the Base64-encoded value has no integrity protection, a normal user can use the provided performbase64.py script to encode 2:admin, replace the cookie value, and obtain administrative access.

Recommendations

- Store only an opaque session identifier in the browser and keep authorization state server-side.
- Use signed, Secure, HttpOnly, and SameSite framework-managed cookies.
- Resolve permissions from trusted server-side state on every sensitive request.
- Invalidate existing sessions after correcting the authorization design.

VWA26062604 - Cookie Role Tampering - Steps

Steps to Reproduce

1. Log in as a normal user and inspect the authInfo cookie in browser Developer Tools.

2. Use the provided performbase64.py script to decode the cookie and confirm that it represents 2:user.

3. Use performbase64.py to encode 2:admin; the resulting value is MjphZG1pbg==.

4. Replace only the authInfo cookie value with MjphZG1pbg== and refresh.

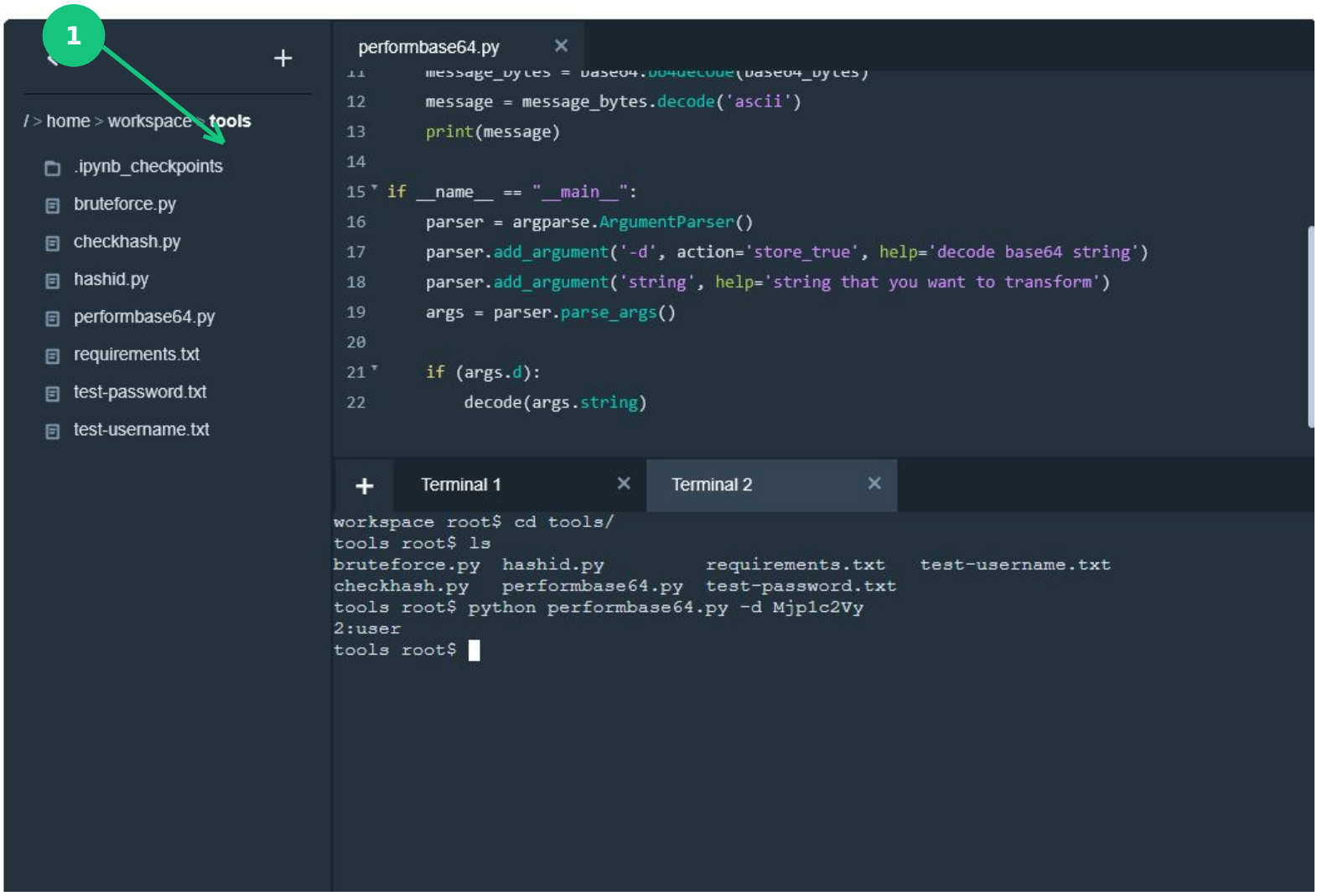
5. Open https://hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/ and confirm that administrative customer data is accessible.

Evidence reference: Evidence pages show the permitted Base64 script and the resulting administrative access.

Assessment used the supplied scripts, browser, and browser Developer Tools only.

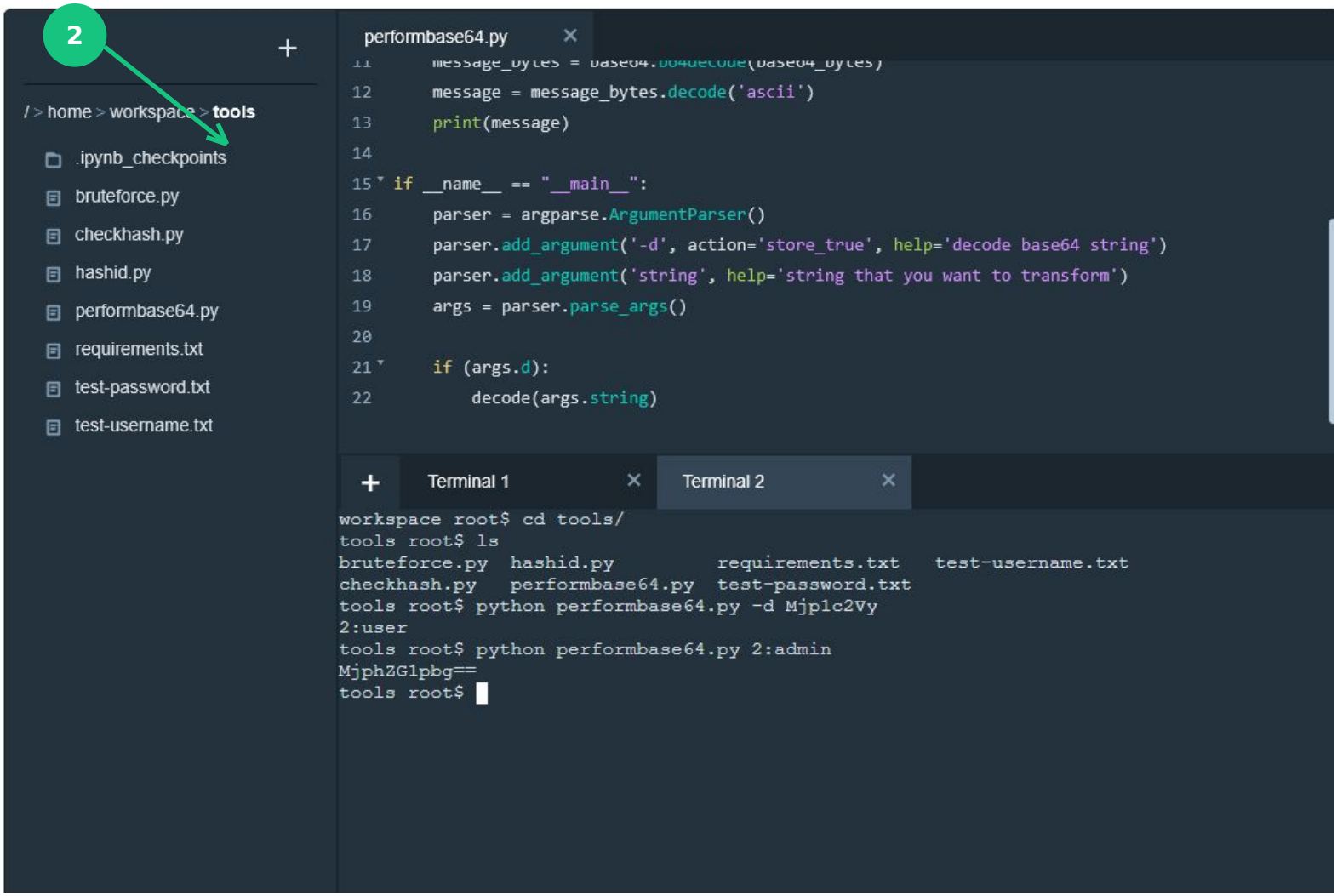
Broken Access

EVIDENCE 1: performbase64.py decodes the normal-user role



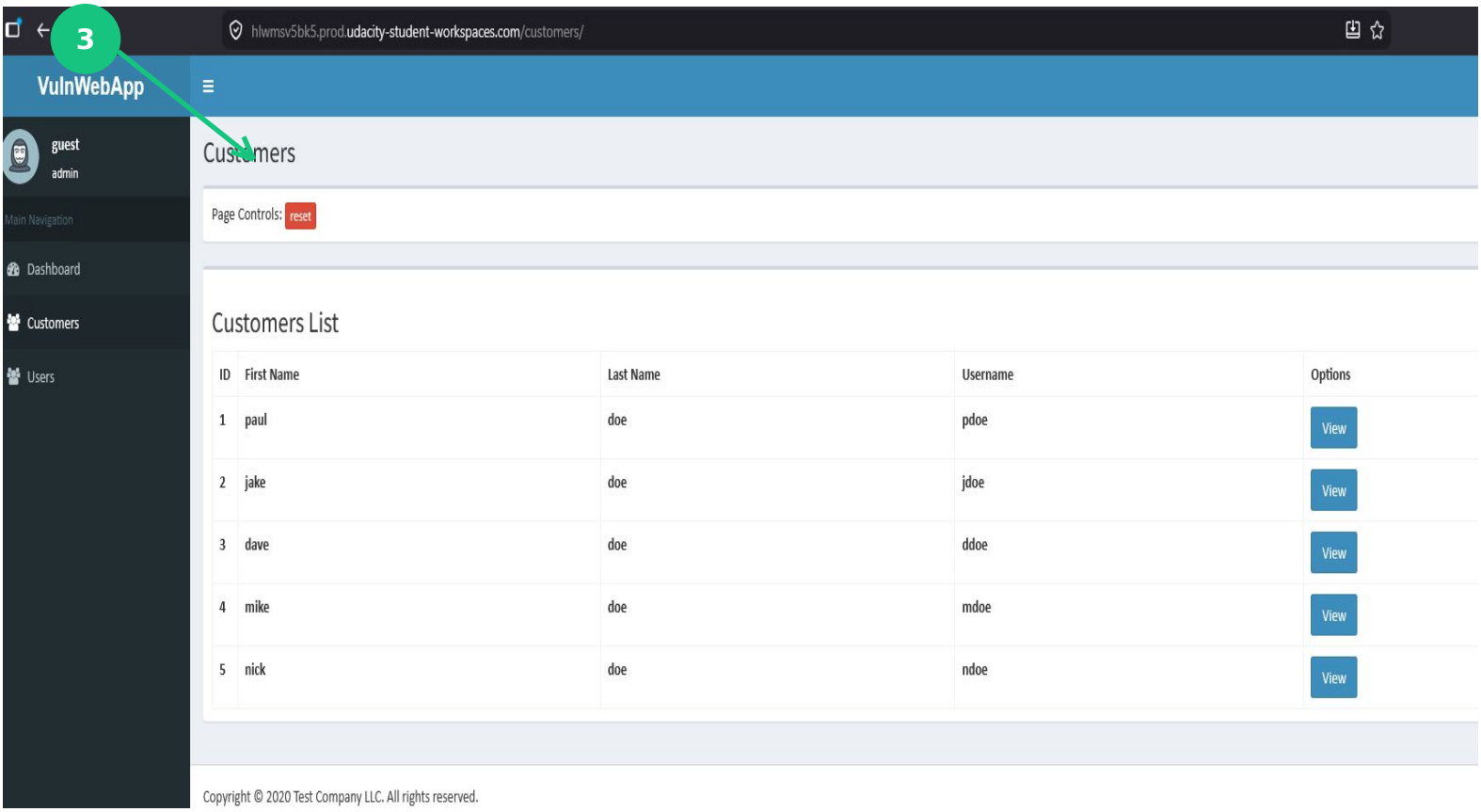
Broken Access

EVIDENCE 2: performbase64.py encodes the administrator role



Broken Access

EVIDENCE 3: Modified role cookie grants customer administration access



VWA26062605 - Sensitive Data Exposure - High

Vulnerability Exploited:	*Sensitive Data Exposure*
Severity:	*High*
OWASP TOP 10 reference:	*A02:2021 - Cryptographic Failures*

Vulnerability Explanation

The customer detail endpoint returns every database column, including an MD5 password hash. Source files also contain database credentials and an application secret key. Disclosure of these values increases the impact of unauthorized API access and source-code exposure.

Recommendations

- Return explicit safe fields through response schemas; never return password hashes.
- Replace MD5 password storage with Argon2id, bcrypt, or PBKDF2 using current parameters.
- Move secrets to protected deployment configuration and rotate exposed values.
- Apply least privilege to the application database account.

VWA26062605 - Sensitive Data Exposure - Steps

Steps to Reproduce

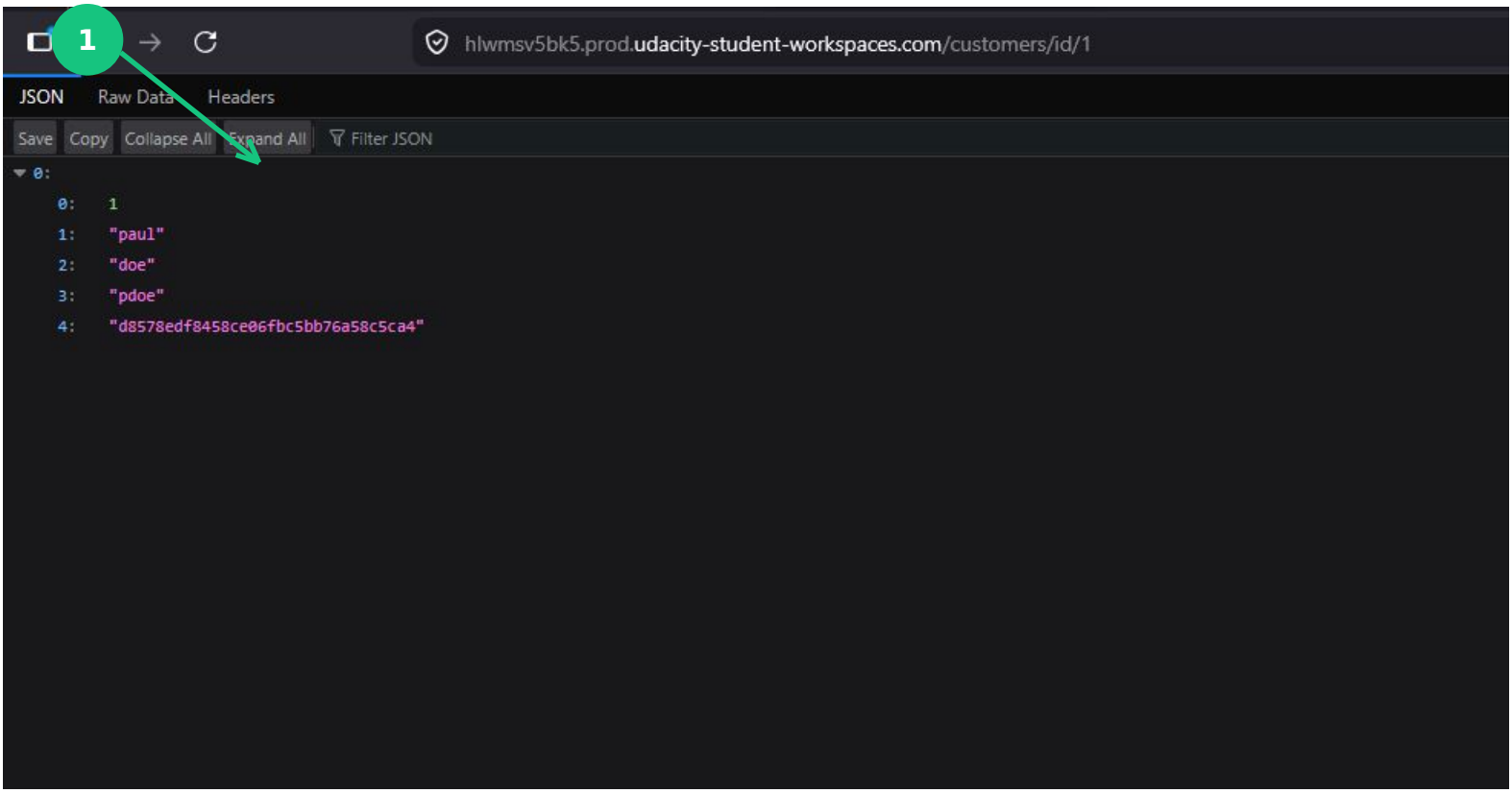
1. Authenticate in the lab and open the customer detail endpoint in the browser.
2. Navigate to https://hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/id/1.
3. Inspect the JSON response and identify the returned username and password-hash fields.
4. Review Site/db/__init__.py and Site/__init__.py in the supplied source.
5. Confirm that database credentials and the Flask secret key are stored directly in source.

Evidence reference: Evidence pages show the password hash returned in browser and Developer Tools responses.

Assessment used the supplied scripts, browser, and browser Developer Tools only.

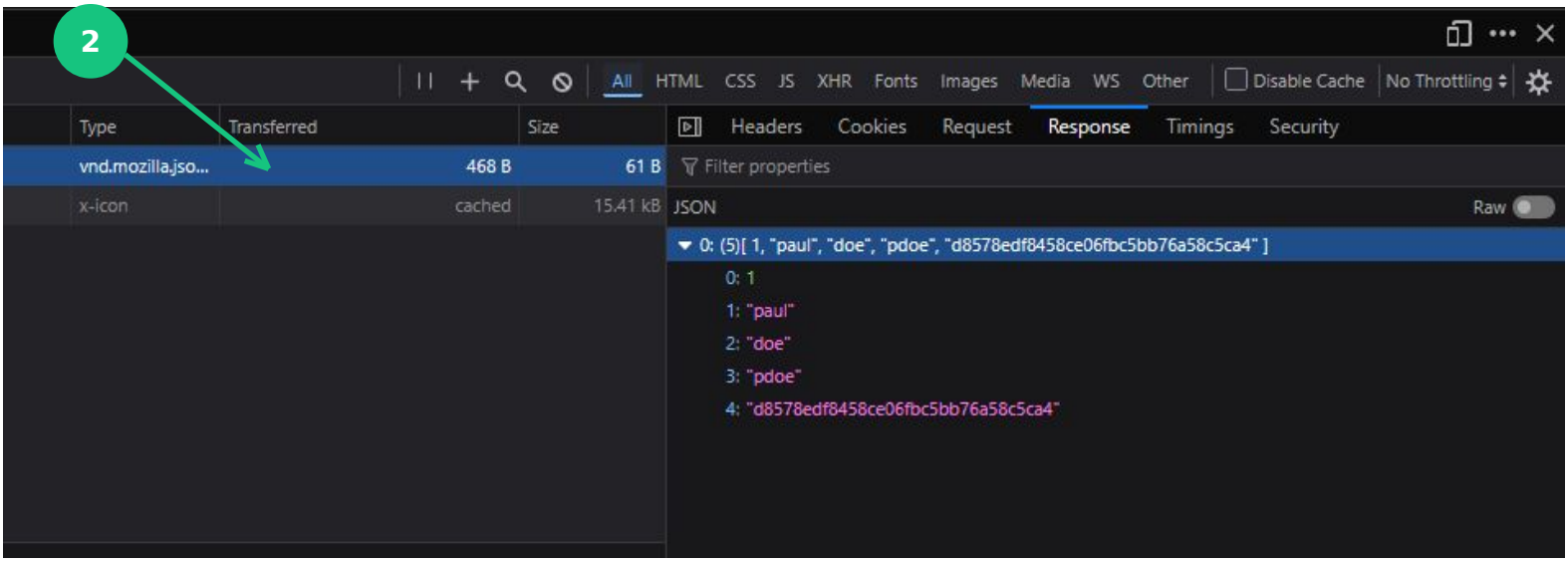
Sensitive Data Exposure

EVIDENCE 1: Customer API response exposes an MD5 password hash



Sensitive Data Exposure

EVIDENCE 2: Developer Tools confirms the sensitive response fields



VWA26062606 - SQL Injection - High

Vulnerability Exploited:	*SQL Injection*
Severity:	*High*
OWASP TOP 10 reference:	*A03:2021 - Injection*

Vulnerability Explanation

The profile user-list endpoint concatenates a route value directly into a SQL statement. Supplying true and false Boolean expressions in the browser address changes the returned result set, demonstrating attacker control over query logic.

Recommendations

- Use parameterized queries for all database operations.
- Constrain identifiers to the expected type at route and service boundaries.
- Use a maintained ORM or safe query builder where practical.
- Apply least-privilege database permissions and add injection regression tests.

VWA26062606 - SQL Injection - Steps

Steps to Reproduce

1. Log in as a normal user.

2. Open the baseline endpoint: https://hlwmsv5bk5.prod.udacity-student-workspaces.com/profile/userlist/1.

3. In the browser address bar, append a URL-encoded false condition: %27%20AND%20%271%27%3D%272.

4. Record the returned JSON, then replace it with the true condition: %27%20OR%20%271%27%3D%271.

5. Compare the baseline, false-condition, and true-condition responses. Confirm that the predicates change the result set.

Evidence reference: The annotated browser screenshot records the baseline response used for the Boolean comparison.

Assessment used the supplied scripts, browser, and browser Developer Tools only.

SQLi

EVIDENCE 1: Baseline user-list response recorded in the browser